claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_0febe894-ff9\agent-ac3fd4b1d5f57f3cb.jsonl`
- SHA-256: `1fea1b9f642b74e95481331c95062213975740acc9c7c09c2879a7dde5f405b7`
- Source modified: `2026-07-07T03:44:29+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_0febe894-ff9`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-07T03:41:25.970Z)

Reviewing GitHub PR #58 "docs: refresh current project documentation" (branch agent/docs-sweep -> main). DOCS-ONLY refresh + a few doc-accuracy source edits (help text, docstrings, Dockerfile comment, pyproject description). +782/-787, 14 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr58.
This is an ACCURACY review: the docs must match the MERGED code (Tracks B-F + ADR 0012 F1-F7 are all merged). Read the doc diff:
  git -C C:/Users/avidu/Projects/Telegram diff 7fadffb..origin/agent/docs-sweep -- <file>
and cross-check every concrete claim against the actual code (src/tg_video_filter/*.py, docs/adr/*). Read code via Read/Grep; run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check defaults/behavior.

KEY GROUND TRUTH (verified by the main reviewer) to check the docs against:
- Selective scanning is OPT-IN: new sources default watched=false (should_ingest_source); the watched flag is user-controlled (set_source_watched / /watch,/unwatch);
  upsert_source no longer clobbers watched (a fix in #41). Commands: /channels, /watch <id|@name>, /unwatch, /watchlist.
- FilterConfig exclusion DEFAULTS: exclude_gif=True, exclude_round=True, exclude_nosound=False (i.e. GIF/round excluded, nosound allowed). The old "all off by default" was WRONG.
- Policy pipeline stages (evaluate_pipeline order): topic-preferences -> url-reputation -> caption-safety -> deep-inspection. Deep inspection is a budget BOUNDARY only; NO production fetcher is wired (ctx.fetcher/budget are None in the live path), so enabling deep_inspect drops items.
- Config lives in policies.config_json (raw dict); save_filter_config MERGES flat FilterConfig keys so ADR 0012 keys (topics/security_checks/deep_inspect) survive a /set. There is NO /set command to set those policy keys yet (dormant).
- Tables: accounts, telegram_sessions, sources, content_items, policies, destinations, routing_rules, delivery_attempts, policy_decisions (+ *_deprecated). policy_decisions has NO TTL/retention cleanup (deferred, issue #56).
- Dedup: 3-tier (message instance, Telegram document.id, metadata fingerprint) via content_items.dedup_key. Delivery modes forward/link/both to me/@username/chat id.
- /why <content_item_id> is account-scoped (get_content_item_for_account), renders the stored policy trace or a caveated metadata replay; markdown-escaped.

Ground rules: READ the doc diff AND the code, cite file:line. Report ONLY doc claims in THIS PR diff that are FACTUALLY WRONG or MISLEADING vs the merged code (a stale table name, a command that does not exist, a wrong default, a wrong ADR status, a "does X" the code does not do). No style nitpicks unless misleading.
Severity: high (a wrong claim a user/operator would act on and be broken by, e.g. wrong command/flag/deploy step), medium (misleading/ambiguous/stale), low (typo/broken link/minor).

ADVERSARIALLY VERIFY this doc-accuracy finding ? try to REFUTE it. Read the cited doc line AND the code it describes; run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if needed. CONFIRMED only if the doc claim genuinely ships in THIS PR diff and is factually wrong/misleading vs the merged code. REFUTED if the doc is actually correct / the code does do

what it says / it is out of scope. PLAUSIBLE if unsettled. Corrected severity (wrong actionable command/step=high; misleading=medium; typo=low).
FINDING (plan_runbook): Runbook §6 recommends set_delivery_target.py, which errors on every live DB (writes to dropped filter_config table)
severity high | docs/runbook.md:151
summary: The runbook's §6 'Delivery Mode and Target' section tells operators to change the delivery target with `scripts\set_delivery_target.py <account_id> <target>`, but that script issues `UPDATE filter_config ... WHERE user_id = ?` against a table that no longer exists on any migrated database, so the documented step fails hard.
why_wrong: Under ADR 0011 (migration 002/003) the legacy `filter_config` table is renamed to `filter_config_deprecated` and init_db() explicitly DROPs any recreated `filter_config` (db.py:230); delivery config now lives in `policies.config_json` and is read via the load_filter_config shim (db.py:1887-1899). Every live DB is at user_version=5, so `filter_config` is gone. The helper script (scripts/set_delivery_target.py:47-51, 64) still runs raw SQL against `filter_config`. An operator following the runbook to change delivery mode/target cannot do so through the documented path.
evidence: Reproduced against a fresh migrated DB (user_version=5): the script's exact SQL `UPDATE filter_config SET config_json=json_set(...,'$.delivery_target',?) WHERE user_id=?` raises `sqlite3.OperationalError: no such table: filter_config`. Table listing after init_db: filter_config present? False; filter_config_deprecated present? True; policies present? True; user_version: 5. Script source: scripts/set_delivery_target.py:47-51. Runbook instruction: docs/runbook.md:151-154 ('Use the helper script for targets: ...set_delivery_target.py <account_id> <target>').
rec: Fix the helper script to update `policies.config_json` for the default policy (matching db.save_filter_config: get_default_policy + save_policy_config, WHERE account_id=? AND is_default=1) instead of `filter_config`. Until the script is fixed, the runbook should not present it as the way to change the delivery target; document the working `UPDATE policies SET config_json=json_set(...,'$.delivery_target',?) WHERE account_id=? AND is_default=1` SQL (parallel to the §8 policy-editing examples) instead. Note: `resolve_target.py` is unaffected (it uses only session files), and the `<account_id>` vs script's `<user_id>` label difference is harmless since accounts.id == telegram user id == session-file id (bootstrap.py:65).

2. assistant (2026-07-07T03:41:28.327Z)

I'll verify this finding by examining the runbook instruction and the script it recommends, then checking against the actual code.

3. user (2026-07-07T03:41:30.902Z)

```
diff --git a/docs/runbook.md b/docs/runbook.md
index b2e3ad7..030c318 100644
--- a/docs/runbook.md
+++ b/docs/runbook.md
@@ -1,73 +1,116 @@
 # Runbook: Telegram Video Filter Userbot

-Operational guide for deploying, onboarding users, and handling common issues.
-See [`design.md`](./design.md) for architecture and [`plan.md`](./plan.md) for
-build status.
+Operational guide for deployment, onboarding, source control, filtering,
+backfill, explainability, and troubleshooting.

 ## 1. Prerequisites

 - A Telegram account you control.
-- `api_id` and `api_hash` from <https://my.telegram.org> ? *API development
-  tools*. Create an app (any name/desc) to get these.
-- Docker + Docker Compose (recommended), **or** Python 3.11+.
-- A persistent directory mounted at `/data` (holds the SQLite DB + session
-  files ? treat as a credential).
-- **Optional but recommended**: install `cryptg` (`pip install cryptg`) for
-  native crypto acceleration. Without it Telethon falls back to slower
-  pure-Python encryption (you'll see a warning at startup). For Docker, add
-  `cryptg` to the image.
-  > **_Note:_** `cryptg` is included as a project dependency by default
```

```
-  > (`pyproject.toml`); you should already have it installed.
-
-## 2. First-time deploy (Docker)
+- `api_id` and `api_hash` from <https://my.telegram.org>.
+- Docker + Docker Compose, or Python 3.11+.
+- A persistent `data/` directory for SQLite and Telethon session files.
+- Optional local virtualenv for scripts/tests:
+
+```powershell
+python -m venv .venv
+.\.venv\Scripts\python.exe -m pip install -e .[dev]
+```
+
+`cryptg` is a runtime dependency and should already be installed through the
+project package.
+
+## 2. First Deploy

 ```bash
-# 1. clone
 git clone https://github.com/avidullu/telegram-video-filter.git
 cd telegram-video-filter

-# 2. configure env
 cp .env.example .env
-#   edit .env: set TG_API_ID, TG_API_HASH, DB_PATH=/data/app.db
-
-# 3. (optional) pre-create the data volume
-mkdir -p data
+# edit .env: TG_API_ID, TG_API_HASH, DB_PATH, SESSIONS_DIR

-# 4. build + run in background
 docker compose up -d --build
 docker compose logs -f app
 ```

-Expected startup log: `manager_started {users: 0}` until you onboard a user.
+Expected first log state: the manager starts with no runnable sessions until a
+user is bootstrapped.

-## 3. Onboard a user (interactive, once per user)
+## 3. Onboard a Telegram User Session

-v1 onboards via a host-side CLI that performs the interactive Telegram login and
-writes a session file into `data/sessions/`.
+Run the bootstrap command in the same environment that can write to `data/`:

 ```bash
-# from the repo root, inside the same env/container that runs the app
 python -m tg_video_filter.bootstrap
-# ? prints the banner (session dir + db path)
-# ? prompts: phone (E.164), then the login code Telegram sends you,
-#            then 2FA password if enabled.
-# ? creates data/sessions/user_{id}.session
-# ? inserts a users row (enabled=1) + default filter_config
 ```

-After onboarding, restart the container so the manager picks up the new user:
+It prompts for:
+
+1. phone number,
+2. Telegram login code,
+3. 2FA password if enabled.
+
+It then creates:
```

+
+- `accounts` row,
+- `telegram_sessions` row,
+- legacy-compatible `users` view data,
+- default policy/config rows,
+- `data/sessions/user_{id}.session`.
+
+Restart the manager after onboarding:

 ```bash
 docker compose restart app
-# ? logs: client_connected {user_id: <id>}
 ```

-> ?? **The session file is a credential.** Anyone with `user_{id}.session` can
-> act as that Telegram account. Keep `data/` private and backed up.
+Session files are credentials. Back up and protect `data/`.
+
+## 4. Live Source Scanning
+
+Live ingestion is opt-in per source.
+
+Unknown sources are lazily discovered as `watched=false`; they are listed but
+not ingested until you watch them. Send commands to your own Saved Messages:
+
+```text
+/channels
+/watch <id|@name|title-fragment>
+/unwatch <id|@name|title-fragment>
+/watchlist
+```
+
+Typical flow:
+
+1. Let new traffic arrive, or run `/backfill` to discover historical dialogs.
+2. Run `/channels`.
+3. Run `/watch -1001234567890` or `/watch Some Channel`.
+4. Confirm with `/watchlist`.
+
+## 5. Filter Commands
+
+These commands persist config changes and hot-reload the in-memory filter
+engine; no restart is needed for these fields.
+
+```text
+/show config
+/show commands
+/set size 10MB
+/set length 15m
+/set max length 15m
+/set min length 5s
+/set min resolution 720p
+/set min res 720p
+/set exclude gif on
+/set exclude round off
+/set exclude nosound on
+```
+
+Units:

-## 4. Adjusting filters
+- size: `KB`, `MB`, `GB`, `TB`; default is MB.
+- duration: `s`, `m`, `h`; default is seconds.

-Filters are stored per-user as JSON in `filter_config`. Default v1 config:

```
+Default flat filter config for fresh installs:

 ```json
 {
@@ -75,224 +118,219 @@ Filters are stored per-user as JSON in `filter_config`. Default v1
config:
   "max_duration_s": 7200,
   "target_height": 1080,
   "max_size_mb": 2048,
+  "exclude_gif": true,
+  "exclude_round": true,
+  "exclude_nosound": false,
   "delivery_mode": "forward",
   "delivery_target": "me"
 }
 ```

-To change a user's config (no v1 UI yet ? use sqlite3 directly):
+## 6. Delivery Mode and Target

-```bash
-sqlite3 data/app.db
-# find the user
-SELECT id, phone FROM users;
-# update (example: allow up to 4h, drop min duration)
-UPDATE filter_config
-   SET config_json = json('{"min_duration_s":0,"max_duration_s":14400,"target_height":1080,"max
_size_mb":2048,"delivery_mode":"forward"}')
- WHERE user_id = <id>;
-.quit
-docker compose restart app
-```
+Delivery mode and target are stored in policy/config JSON. There is no
+Saved-Messages command for changing them yet, so update deliberately and restart
+the manager.

-### Backfill historical videos (v1.1)
+Modes:

-?? Backfill scans **all** channels for historical videos. On large channels
-it can run for minutes and may trigger Telegram rate limits. Progress is
-reported in real time to your Saved Messages.
+| Mode | Behavior |
+|---|---|
+| `forward` | Forward the Telegram video message. |
+| `link` | Send a `t.me` link plus metadata summary; falls back to forward if no link exists. |
+| `both` | Forward and send the link summary. |

-While the manager is running, send a message to your **Saved Messages**:
+Targets:

+| Target | Meaning |
+|---|---|
+| `"me"` | Saved Messages. |
+| `"@your-feed"` | Public channel or chat username. |
+| `"-1001234567890"` | Numeric Telegram chat/channel id. |
+
+Use the helper script for targets:
+
+```powershell
+.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
 ```
-/backfill        # scan last 90 days (default)
-/backfill 7d     # scan last 7 days
-/backfill 8h     # scan last 8 hours
```
```

-/backfill 2w   # scan last 2 weeks
-/backfill 30   # scan last 30 days
-```

-The bot responds:
+If Telethon cannot resolve a target, refresh the entity cache:
+
+```powershell
+.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
 ```
-? Starting backfill of all channels (last 90 days)...
-? Scanning #ChannelName (last 90 days)...
-   #ChannelName: 142 videos, 8 matched so far...
-? #ChannelName: 156 videos, 11 matched
-? Backfill complete: 5 channels scanned, 823 videos found, 47 matched
+
+Then restart:
+
+```bash
+docker compose restart app
 ```

-- Videos already seen (by live watcher or prior backfill) are **skipped**.
-- Matches are delivered according to your current `delivery_mode`.
-- If a channel triggers a flood wait, it sleeps and continues.
+Backfill progress messages always go to Saved Messages; delivered matches honor
+`delivery_target`.

-### Tuning filter criteria live (v1.3)
+## 7. Backfill

-While the manager is running, send these commands to **Saved Messages**:
+Backfill scans historical channel/group dialogs and can be expensive on large
+accounts. It does not use the live `watched` gate; run it intentionally.

-```
-/show config   # display current filter settings
-/show commands   # list all available commands
-/set size 10MB   # set max file size
-/set length 15m   # set max video duration
-/set max length 15m   # same as above (alias)
-/set min length 5s   # set min video duration
-/set exclude gif on   # exclude Telegram GIFs
-/set exclude round on   # exclude circular video messages
-/set exclude nosound on   # exclude videos without audio
+```text
+/backfill
+/backfill 8h
+/backfill 7d
+/backfill 2w
+/stop
 ```

-All changes persist to the DB and take effect immediately ? no restart needed.
-Exclusions are **off by default** for backwards compatibility.
+Behavior:

-### Switching delivery mode (v1.1)
+- default window is 90 days;
+- dedup skips already-seen content;
+- matching content is delivered with current delivery mode/target;
+- progress is reported to Saved Messages;
+- flood waits are slept through and logged.

-The ``delivery_mode`` field controls how matched videos are delivered to your
 ```

```
-Saved Messages:
+## 8. Policy Pipeline

-| Mode | What you get |
-|---|---|
-| ``forward`` (default) | The video itself is forwarded (re-uploaded). |
-| ``link`` | A text message with a ``t.me`` link + metadata summary (resolution, duration,
size). No re-upload. |
-| ``both`` | Forward the video AND send the link message. |
+Metadata filters run first. If they pass, ADR 0012 policy stages run in order:

-Example ? switch to link-only mode:
+1. topic preferences,
+2. URL/domain reputation,
+3. caption safety,
+4. deep inspection boundary.
```

4. user (2026-07-07T03:41:30.909Z)

```
1       """One-shot helper: set delivery_target for a user.
2
3       Usage (run from the repo root with the VENV python)::
4
5           # Windows (PowerShell)
6           .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
7           # macOS / Linux
8           .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
9
10      Examples::
11
12          .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py 8010202291
-1003957060579
13          .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py 8010202291 @my-channel
14          .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py 8010202291 me
15
16      Note: using the bare ``python`` from your PATH will fail with
17      ``ModuleNotFoundError: No module named 'pydantic_settings'`` because the
18      project's dependencies live in ``.venv``, not the system interpreter.
19
20      Reads the DB path from app settings (``Settings.db_path``) so it respects
21      .env / TG_DB_PATH the same way the manager does.
22      """
23
24      from __future__ import annotations
25
26      import sqlite3
27      import sys
28
29
30      def main() -> int:
31          if len(sys.argv) != 3:
32              print("Usage: python scripts/set_delivery_target.py <user_id> <target>",
file=sys.stderr)
33              print("  target: 'me', '@username', or a chat id like -1001234567890",
file=sys.stderr)
34              return 2
35
36          user_id = int(sys.argv[1])
37          target = sys.argv[2].strip()
38
39          # Read db_path from settings so .env / env vars are honoured.
40          sys.path.insert(0, "src")
41          from tg_video_filter.config import get_settings
42
43          db_path = get_settings().db_path
```

```
44
45          conn = sqlite3.connect(db_path)
46          try:
47              cur = conn.execute(
48                  "UPDATE filter_config "
49                  "SET config_json = json_set(config_json, '$.delivery_target', ?) "
50                  "WHERE user_id = ?",
51                  (target, user_id),
52              )
53              conn.commit()
54              if cur.rowcount == 0:
55                  print(f"ERROR: no filter_config row for user_id={user_id}.",
file=sys.stderr)
56                  print("Run `python -m tg_video_filter.bootstrap` first.", file=sys.stderr)
57                  return 1
58          finally:
59              conn.close()
60
61          # Show the resulting config so you can verify.
62          conn = sqlite3.connect(db_path)
63          row = conn.execute(
64              "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
65          ).fetchone()
66          conn.close()
67
68          print(f"? delivery_target for user {user_id} set to: {target}")
69          print(f"  current config: {row[0] if row else '(none)'}")
70          print("\nRestart the manager for the change to take effect:")
71          print("  python -m tg_video_filter        # or")
72          print("  docker compose restart app")
73          return 0
74
75
76      if __name__ == "__main__":
77          sys.exit(main())
78
```

5. assistant (2026-07-07T03:41:33.352Z)

Now let me verify the ground truth: check db.py for the filter_config table drop and confirm the table doesn't exist on a migrated DB.

6. user (2026-07-07T03:41:35.287Z)

```
86:CREATE TABLE IF NOT EXISTS filter_config (
222:    # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
230:        await conn.execute("DROP TABLE IF EXISTS filter_config")
372:    data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
378:    FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
440:    # Rebuild channels / videos / filter_config so their FK constraints point
487:            "filter_config",
489:            CREATE TABLE filter_config (
514:    # filter_config tables were already rebuilt above with FKs to accounts(id),
527:    ``channels`` / ``videos`` / ``filter_config`` tables and renames the
632:    # 3. Backfill policies from filter_config: one 'default' policy per account
633:    #    (Option 2A). is_default=1 so the legacy load_filter_config shim reads
642:        FROM filter_config
679:    await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
687:    ``filter_config_deprecated.delivery_target`` value, then creates a default
700:    if not await _table_exists(conn, "filter_config_deprecated"):
703:    # 1. Backfill default destinations from filter_config_deprecated.
725:        FROM filter_config_deprecated
745:            FROM filter_config_deprecated fcd
1374:    Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1887:async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
```

```
1890:    ADR 0011 (Option 2A): ``filter_config`` is renamed to
1891:    ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1902:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
2016:    "load_filter_config",
2017:    "save_filter_config",
```

7. assistant (2026-07-07T03:41:37.066Z)

Let me look at the relevant db.py lines and reproduce the failure against a migrated DB.

8. user (2026-07-07T03:41:38.580Z)

```
218         await conn.execute("PRAGMA foreign_keys = ON")
219         await conn.execute("PRAGMA busy_timeout = 5000")
220         await conn.executescript(_SCHEMA)
221         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
222         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
223         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
224         # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
225         # any *empty* recreations of those legacy tables from a subsequent
226         # ``init_db`` are dropped here so they don't shadow the renamed copies.
227         if await _table_exists(conn, "channels_deprecated"):
228             await conn.execute("DROP TABLE IF EXISTS channels")
229             await conn.execute("DROP TABLE IF EXISTS videos")
230             await conn.execute("DROP TABLE IF EXISTS filter_config")
231         await _apply_migrations(conn)
232         await conn.commit()
233
234
235     # --- migrations ----------------------------------------------------------
```

9. user (2026-07-07T03:43:44.392Z)

Command running in background with ID: bid1u1l8j. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bid1u1l8j.output. You will be notified when it completes. To check interim output, use Read on that file path.

10. assistant (2026-07-07T03:43:46.836Z)

Note the drop only fires when `channels_deprecated` exists (i.e., a legacy DB that migrated). For a fresh DB, migration 003 renames channels to channels_deprecated too. Let me check the background output.

11. user (2026-07-07T03:43:47.974Z)

```
1       Traceback (most recent call last):
2         File "<string>", line 25, in <module>
3           asyncio.run(main())
4           ~~~~~~~~~~~^^^^^^^^
5         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 195, in run
6           return runner.run(main)
7                  ~~~~~~~~~~^^^^^^
8         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 118, in run
9           return self._loop.run_until_complete(task)
10                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
11        File
"C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725,
in run_until_complete
12          return future.result()
13                 ~~~~~~~~~~~~~^^
```

```
14        File "<string>", line 9, in main
15          await db.init_db(conn)
16        File "C:\Users\avidu\Projects\Telegram\src\tg_video_filter\db.py", line 210, in
init_db
17          async with aiosqlite.connect(db_path) as conn:
18                     ~~~~~~~~~~~~~~~~~^^^^^^^^^
19        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 181, in __aenter__
20          return await self
21                 ^^^^^^^^^^
22        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 168, in _connect
23          self._connection = await future
24                             ^^^^^^^^^^^^
25        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 63, in _connection_worker_thread
26          result = function()
27        File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py",
line 466, in connector
28          return sqlite3.connect(loc, **kwargs)
29                 ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^
30      sqlite3.OperationalError: unable to open database file
31
```

12. assistant (2026-07-07T03:43:49.902Z)

```
init_db takes a db_path string, not a connection. Let me fix.
```

13. user (2026-07-07T03:43:52.417Z)

```
(db_path: 'str') -> 'None'
```

14. user (2026-07-07T03:44:00.263Z)

```
user_version 5
filter_config exists? False
filter_config_deprecated exists? True
policies exists? True
script SQL error: OperationalError no such table: filter_config
```

15. assistant (2026-07-07T03:44:03.845Z)

```
Confirmed. On a freshly-initialized DB at user_version=5, `filter_config` does not exist (only
`filter_config_deprecated`), and the script's exact `UPDATE filter_config ... WHERE user_id=?`
raises `no such table: filter_config`.

Now let me confirm the runbook diff in THIS PR presents the script as the way to change delivery
target, and confirm save_filter_config uses policies.
```

16. user (2026-07-07T03:44:06.438Z)

```
174:+## 6. Delivery Mode and Target
212:+Use the helper script for targets:
215:+.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
369:-Instead of hand-writing SQL, use the helper script (run with the **venv** python,
374:-    .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
376:-    .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
```

17. user (2026-07-07T03:44:06.787Z)

```
1902    async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1903        """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1904
```

```
1905        ADR 0012: future pipeline keys live in the same ``policies.config_json``
1906        object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1907        merge the flat config fields into the existing object so unknown policy
1908        keys survive legacy ``/set`` command updates.
1909        """
1910        existing = await get_default_policy(conn, user_id)
1911        raw_config: dict[str, object] = {}
1912        if existing is not None:
1913            try:
1914                raw_config = _policy_config_json_to_dict(existing.config_json)
1915            except (json.JSONDecodeError, ValueError):
1916                log.warning(
1917                    "resetting malformed policy config while saving FilterConfig user=%s "
1918                    "policy_id=%s",
1919                    user_id,
1920                    existing.id,
1921                    exc_info=True,
1922                )
1923                raw_config = {}
1924        raw_config.update(cfg.model_dump(mode="json"))
1925        await save_policy_config(conn, user_id, raw_config)
1926
1927
1928    # --- bulk helpers (used by the manager on startup) ---------------------------
1929
1930
1931    async def list_videos_for_user(  # v2: used by on-demand /search; not called in v1
```

18. assistant (2026-07-07T03:44:11.800Z)

The finding is fully confirmed. Interesting nuance: the OLD runbook (pre-PR) also referenced this same broken script (line 369-376 in the diff shows the removed old version also pointed to `set_delivery_target.py`). But the PR does not fix it ? it re-adds the same broken instruction at line 215 of the diff (runbook.md:151 in the new file). Since this is a doc-accuracy PR that touches and re-presents this exact instruction in the new §6, the wrong claim genuinely ships in this diff.

Let me verify the runbook.md:151 line number in the checked-out file.

19. user (2026-07-07T03:44:13.465Z)

```
151:Use the helper script for targets:
154:.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
160:.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
333:| Delivery target cannot resolve | Telethon session cache lacks entity | Run
`scripts/resolve_target.py`, then restart. |
```

20. assistant (2026-07-07T03:44:22.485Z)

Confirmed at docs/runbook.md:151-154. The finding is accurate.

Summary of verification:
- The PR's new §6 (docs/runbook.md:151-154) instructs operators to run `scripts\set_delivery_target.py <account_id> <target>` to change the delivery target.
- scripts/set_delivery_target.py:47-51 issues `UPDATE filter_config SET config_json=json_set(...,'$.delivery_target',?) WHERE user_id=?`.
- On a migrated/fresh DB at user_version=5, `filter_config` does not exist ? db.py:230 drops it and db.py:679 renamed it to `filter_config_deprecated`. Reproduced: the exact SQL raises `sqlite3.OperationalError: no such table: filter_config`. Table check showed filter_config=False, filter_config_deprecated=True, policies=True, user_version=5.
- Delivery config now lives in policies.config_json (db.py:1902-1925 save_filter_config merges flat FilterConfig keys into the default policy via get_default_policy + save_policy_config).
- The PR touches and re-presents this instruction in the new §6, so the wrong operational step genuinely ships in this diff.

One nuance worth noting for severity calibration: the broken script is pre-existing (the old runbook also referenced it), but this doc-accuracy PR re-documents it as the working path rather than correcting it, so it remains a wrong actionable operator step that ships here. The `resolve_target.py` reference is unaffected (it uses session files only). Severity high is correct: an operator following this documented path to change delivery mode/target hits a hard failure.

21. user (2026-07-07T03:44:29.207Z)

Structured output provided successfully